Assignment #5

2026-06-21

R Markdown

##First I am going to load the data and check the variables

```
library(cluster)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggplot2)
cereals <- read.csv("Cereals.csv")
head(cereals)
```

```
##                         name mfr type calories protein fat sodium fiber carbo
## 1                  100%_Bran   N    C       70       4   1    130  10.0   5.0
## 2          100%_Natural_Bran   Q    C      120       3   5     15   2.0   8.0
## 3                   All-Bran   K    C       70       4   1    260   9.0   7.0
## 4 All-Bran_with_Extra_Fiber   K    C       50       4   0    140  14.0   8.0
## 5             Almond_Delight   R    C      110       2   2    200   1.0  14.0
## 6    Apple_Cinnamon_Cheerios   G    C      110       2   2    180   1.5  10.5
##   sugars potass vitamins shelf weight cups   rating
## 1      6    280       25     3      1 0.33 68.40297
## 2      8    135        0     3      1 1.00 33.98368
## 3      5    320       25     3      1 0.33 59.42551
## 4      0    330       25     3      1 0.50 93.70491
## 5      8     NA       25     3      1 0.75 34.38484
## 6     10     70       25     1      1 0.75 29.50954
```

```
str(cereals)
```

```
## 'data.frame':    77 obs. of  16 variables:
##  $ name    : chr  "100%_Bran" "100%_Natural_Bran" "All-Bran" "All-Bran_with_Extra_Fiber" ...
```

```
##  $ mfr     : chr  "N" "Q" "K" "K" ...
##  $ type    : chr  "C" "C" "C" "C" ...
##  $ calories: int  70 120 70 50 110 110 110 130 90 90 ...
##  $ protein : int  4 3 4 4 2 2 2 3 2 3 ...
##  $ fat     : int  1 5 1 0 2 2 0 2 1 0 ...
##  $ sodium  : int  130 15 260 140 200 180 125 210 200 210 ...
##  $ fiber   : num  10 2 9 14 1 1.5 1 2 4 5 ...
##  $ carbo   : num  5 8 7 8 14 10.5 11 18 15 13 ...
##  $ sugars  : int  6 8 5 0 8 10 14 8 6 5 ...
##  $ potass  : int  280 135 320 330 NA 70 30 100 125 190 ...
##  $ vitamins: int  25 0 25 25 25 25 25 25 25 25 ...
##  $ shelf   : int  3 3 3 3 3 1 2 3 1 3 ...
##  $ weight  : num  1 1 1 1 1 1 1 1.33 1 1 ...
##  $ cups    : num  0.33 1 0.33 0.5 0.75 0.75 1 0.75 0.67 0.67 ...
##  $ rating  : num  68.4 34 59.4 93.7 34.4 ...
```

```
summary(cereals)
```

```
##      name               mfr                type              calories    
##  Length:77          Length:77          Length:77          Min.   : 50.0  
##  Class :character   Class :character   Class :character   1st Qu.:100.0  
##  Mode  :character   Mode  :character   Mode  :character   Median :110.0  
##                                                           Mean   :106.9  
##                                                           3rd Qu.:110.0  
##                                                           Max.   :160.0  
##                                                                          
##     protein           fat            sodium          fiber       
##  Min.   :1.000   Min.   :0.000   Min.   :  0.0   Min.   : 0.000  
##  1st Qu.:2.000   1st Qu.:0.000   1st Qu.:130.0   1st Qu.: 1.000  
##  Median :3.000   Median :1.000   Median :180.0   Median : 2.000  
##  Mean   :2.545   Mean   :1.013   Mean   :159.7   Mean   : 2.152  
##  3rd Qu.:3.000   3rd Qu.:2.000   3rd Qu.:210.0   3rd Qu.: 3.000  
##  Max.   :6.000   Max.   :5.000   Max.   :320.0   Max.   :14.000  
##                                                                  
##      carbo          sugars           potass          vitamins     
##  Min.   : 5.0   Min.   : 0.000   Min.   : 15.00   Min.   :  0.00  
##  1st Qu.:12.0   1st Qu.: 3.000   1st Qu.: 42.50   1st Qu.: 25.00  
##  Median :14.5   Median : 7.000   Median : 90.00   Median : 25.00  
##  Mean   :14.8   Mean   : 7.026   Mean   : 98.67   Mean   : 28.25  
##  3rd Qu.:17.0   3rd Qu.:11.000   3rd Qu.:120.00   3rd Qu.: 25.00  
##  Max.   :23.0   Max.   :15.000   Max.   :330.00   Max.   :100.00  
##  NA's   :1      NA's   :1        NA's   :2                        
##      shelf           weight          cups           rating     
##  Min.   :1.000   Min.   :0.50   Min.   :0.250   Min.   :18.04  
##  1st Qu.:1.000   1st Qu.:1.00   1st Qu.:0.670   1st Qu.:33.17  
##  Median :2.000   Median :1.00   Median :0.750   Median :40.40  
##  Mean   :2.208   Mean   :1.03   Mean   :0.821   Mean   :42.67  
##  3rd Qu.:3.000   3rd Qu.:1.00   3rd Qu.:1.000   3rd Qu.:50.83  
##  Max.   :3.000   Max.   :1.50   Max.   :1.500   Max.   :93.70  
## 
```

##Now I am going to remove missing values

```r
colSums(is.na(cereals))
```

```
##     name       mfr      type calories   protein       fat    sodium     fiber
##        0         0         0        0         0         0         0         0
##    carbo    sugars    potass vitamins     shelf    weight      cups    rating
##        1         1         2        0         0         0         0         0
```

```r
cereals_clean <- na.omit(cereals)
nrow(cereals)
```

```
## [1] 77
```

```r
nrow(cereals_clean)
```

```
## [1] 74
```

##When can not cluster using name, mfr, or type. Those are labels not measurements so we have to select only numeric variables.

```r
cereal_num <- cereals_clean%>%select(where(is.numeric))
head(cereal_num)
```

```
##   calories protein fat sodium fiber carbo sugars potass vitamins shelf weight
## 1       70       4   1    130  10.0   5.0      6    280       25     3      1
## 2      120       3   5     15   2.0   8.0      8    135        0     3      1
## 3       70       4   1    260   9.0   7.0      5    320       25     3      1
## 4       50       4   0    140  14.0   8.0      0    330       25     3      1
## 6      110       2   2    180   1.5  10.5     10     70       25     1      1
## 7      110       2   0    125   1.0  11.0     14     30       25     2      1
##   cups   rating
## 1 0.33 68.40297
## 2 1.00 33.98368
## 3 0.33 59.42551
## 4 0.50 93.70491
## 6 0.75 29.50954
## 7 1.00 33.17409
```

##Now that we have just the numerical data, we have to normalize this numerical data because variables are on different scales and this can mess up the clustering.

```r
cereal_scaled <- scale(cereal_num)
summary(cereal_scaled)
```

```
##     calories          protein              fat               sodium
##  Min.   :-2.8738   Min.   :-1.40687   Min.   :-0.9932   Min.   :-1.9616
##  1st Qu.:-0.3541   1st Qu.:-0.47733   1st Qu.:-0.9932   1st Qu.:-0.3306
##  Median : 0.1498   Median :-0.01256   Median : 0.0000   Median : 0.2131
##  Mean   : 0.0000   Mean   : 0.00000   Mean   : 0.0000   Mean   : 0.0000
##  3rd Qu.: 0.1498   3rd Qu.: 0.45221   3rd Qu.: 0.0000   3rd Qu.: 0.6661
##  Max.   : 2.6695   Max.   : 3.24083   Max.   : 3.9729   Max.   : 1.9045
```

```
##      fiber             carbo             sugars            potass
##  Min.   :-0.89778   Min.   :-2.50014   Min.   :-1.6306   Min.   :-1.1783
##  1st Qu.:-0.79462   1st Qu.:-0.70143   1st Qu.:-0.9424   1st Qu.:-0.8079
##  Median :-0.07249   Median :-0.05903   Median :-0.0248   Median :-0.1201
##  Mean   : 0.00000   Mean   : 0.00000   Mean   : 0.0000   Mean   : 0.0000
##  3rd Qu.: 0.34015   3rd Qu.: 0.58337   3rd Qu.: 0.8928   3rd Qu.: 0.3031
##  Max.   : 4.87925   Max.   : 2.12512   Max.   : 1.8104   Max.   : 3.2660
##     vitamins          shelf             weight             cups
##  Min.   :-1.3032   Min.   :-1.4617   Min.   :-3.4600   Min.   :-2.4251
##  1st Qu.:-0.1818   1st Qu.:-1.1612   1st Qu.:-0.2008   1st Qu.:-0.6432
##  Median :-0.1818   Median :-0.2599   Median :-0.2008   Median :-0.3038
##  Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000   Mean   : 0.0000
##  3rd Qu.:-0.1818   3rd Qu.: 0.9420   3rd Qu.:-0.2008   3rd Qu.: 0.7568
##  Max.   : 3.1822   Max.   : 0.9420   Max.   : 3.0583   Max.   : 2.8780
##      rating
##  Min.   :-1.7336
##  1st Qu.:-0.7071
##  Median :-0.1510
##  Mean   : 0.0000
##  3rd Qu.: 0.5807
##  Max.   : 3.6578
```

##Now that the numerical data is normalized, we need to calculate the distance of each cereal data point. This measures how far apart each cereal is from every other cereal. And if the cereals are close together they must be nutriionally similiar. We use Euclidean distance.

```
cereal_dist <- dist(cereal_scaled,method="euclidean")
```

##Now we are going to test different linkage method and choose the one with the strongest clustering structure.

```
methods <- c("single", "complete", "average", "ward")

agnes_results <- data.frame(
  Method = methods,
  Agglomerative_Coefficient = sapply(methods, function(m) {
    agnes(cereal_scaled, method = m)$ac
  })
)

agnes_results
```

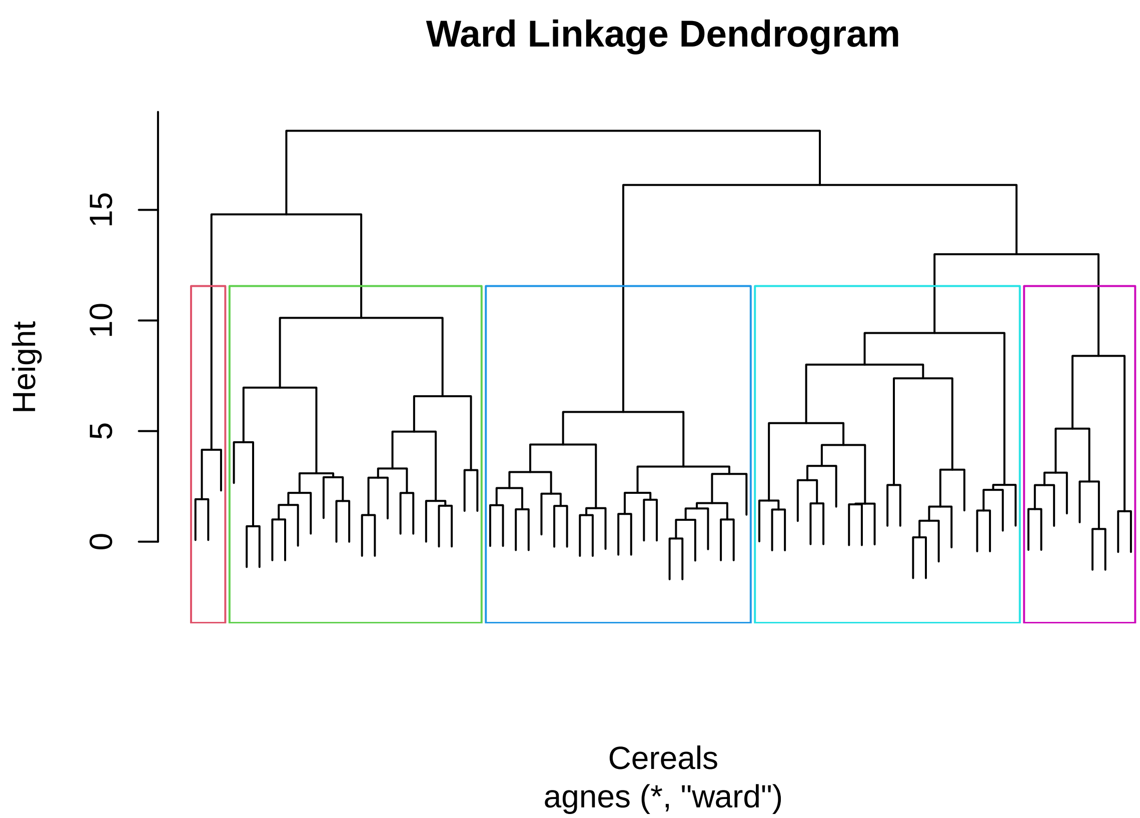
```
##            Method Agglomerative_Coefficient
## single     single                 0.6067859
## complete complete                 0.8353712
## average   average                 0.7766075
## ward         ward                 0.9046042
```

```
agnes_ward <- agnes(cereal_scaled, method = "ward")
```

The ward linkage has the highest agglomerative coefficient of about .90, meaning it gives the best and strongest hiercrchial clustering structure compared to the other linkage methods. Therefore it will be selected for the final hierarchical clustering analysis.

Now that we have chose ward, we can create the dendrogram. This is where we can visually see the hierarchical clustering tree

```
agnes_ward <- agnes(cereal_scaled,method="ward")
hc_ward <- as.hclust(agnes_ward)
plot(hc_ward,labels=FALSE,main="Ward Linkage Dendrogram",xlab="Cereals",ylab="Height")
rect.hclust(hc_ward,k=5,border=2:6)
```



By looking at the tree we can see each cereal begins as its own cluster and similiar cereals are merging together as the height increases.

##We got the tree, now its time to cut the tree into 5 groups and save those group numbers into the dataset

```
k <- 5
cereals_clean$Cluster <- cutree(hc_ward,k=k)
table(cereals_clean$Cluster)
```

```
##
##  1  2  3  4  5
##  3 20 21 21  9
```

##Now lets get the average nutrition values for each cluster

```r
cluster_profile <- cereals_clean %>%
  group_by(Cluster) %>%
  summarise(
    Count = n(),
    Avg_Calories = mean(calories),
    Avg_Protein = mean(protein),
    Avg_Fat = mean(fat),
    Avg_Sodium = mean(sodium),
    Avg_Fiber = mean(fiber),
    Avg_Carbs = mean(carbo),
    Avg_Sugars = mean(sugars),
    Avg_Potass = mean(potass),
    Avg_Rating = mean(rating)
  )

cluster_profile
```

```
## # A tibble: 5 x 11
##   Cluster Count Avg_Calories Avg_Protein Avg_Fat Avg_Sodium Avg_Fiber Avg_Carbs
##     <int> <int>        <dbl>       <dbl>   <dbl>      <dbl>     <dbl>     <dbl>
## 1       1     3         63.3        4      0.667      177.     11          6.67
## 2       2    20        124          3.15   1.95       155       3.1       14.0
## 3       3    21        111.         1.52   1          172.      0.571     12.6
## 4       4    21        104.         2.71   0.524      226.      1.67      18.5
## 5       5     9         82.2        2.44   0.111        1.67    2.11      15.3
## # i 3 more variables: Avg_Sugars <dbl>, Avg_Potass <dbl>, Avg_Rating <dbl>
```

It seems like cluster 1 is the healthiest cluster becaue it has the highest fiber and highest rating. Cluster 5 on the other hand is the best practical cafeteria choice because it has more cereals than cluster 1 and has very low calories, fat, sodium, and suger.

##I want to see the list of cereals in each cluster so I can specifically see the healthier cluster

```r
cereals_clean %>%
  select(name, Cluster, calories, protein, fat, sodium, fiber, sugars, potass, rating) %>%
  arrange(Cluster, name)
```

```
##                                       name Cluster calories protein fat sodium
## 1                                100%_Bran       1       70       4   1    130
## 2                                 All-Bran       1       70       4   1    260
## 3                All-Bran_with_Extra_Fiber       1       50       4   0    140
## 4                        100%_Natural_Bran       2      120       3   5     15
## 5                                  Basic_4       2      130       3   2    210
## 6                                 Clusters       2      110       3   2    140
## 7                       Cracklin'_Oat_Bran       2      110       3   3    140
## 8                     Crispy_Wheat_&_Raisins       2      100       2   1    140
## 9  Fruit_&_Fibre_Dates,_Walnuts,_and_Oats       2      120       3   2    160
## 10                            Fruitful_Bran       2      120       3   0    240
## 11                      Great_Grains_Pecan       2      120       3   3     75
## 12                    Just_Right_Fruit_&_Nut       2      140       3   1    170
## 13                                    Life       2      100       4   2    150
## 14          Muesli_Raisins,_Dates,_&_Almonds       2      150       4   3     95
## 15         Muesli_Raisins,_Peaches,_&_Pecans       2      150       4   3    150
```

```
## 16            Mueslix_Crispy_Blend    2    160     3    2    150
## 17      Nutri-Grain_Almond-Raisin    2    140     3    2    220
## 18           Oatmeal_Raisin_Crisp    2    130     3    2    170
## 19          Post_Nat._Raisin_Bran    2    120     3    1    200
## 20             Quaker_Oat_Squares    2    100     4    1    135
## 21                    Raisin_Bran    2    120     3    1    210
## 22                Raisin_Nut_Bran    2    100     3    2    140
## 23              Total_Raisin_Bran    2    140     3    1    190
## 24        Apple_Cinnamon_Cheerios    3    110     2    2    180
## 25                    Apple_Jacks    3    110     2    0    125
## 26                    Cap'n'Crunch    3    120     1    2    220
## 27          Cinnamon_Toast_Crunch    3    120     1    3    210
## 28                    Cocoa_Puffs    3    110     1    1    180
## 29                      Corn_Pops    3    110     1    0     90
## 30                  Count_Chocula    3    110     1    1    180
## 31                    Froot_Loops    3    110     2    1    125
## 32                 Frosted_Flakes    3    110     1    0    200
## 33                 Fruity_Pebbles    3    110     1    1    135
## 34                   Golden_Crisp    3    100     2    0     45
## 35                 Golden_Grahams    3    110     1    1    280
## 36                     Honey-comb    3    110     1    0    180
## 37               Honey_Graham_Ohs    3    120     1    2    220
## 38             Honey_Nut_Cheerios    3    110     3    1    250
## 39                   Lucky_Charms    3    110     2    1    180
## 40           Multi-Grain_Cheerios    3    100     2    1    220
## 41               Nut&Honey_Crunch    3    120     2    1    190
## 42                         Smacks    3    110     2    1     70
## 43                           Trix    3    110     1    1    140
## 44             Wheaties_Honey_Gold    3    110     2    1    200
## 45                      Bran_Chex    4     90     2    1    200
## 46                    Bran_Flakes    4     90     3    0    210
## 47                       Cheerios    4    110     6    2    290
## 48                      Corn_Chex    4    110     2    0    280
## 49                    Corn_Flakes    4    100     2    0    290
## 50                        Crispix    4    110     2    0    220
## 51                    Double_Chex    4    100     2    0    190
## 52                     Grape-Nuts    4    110     3    0    170
## 53              Grape_Nuts_Flakes    4    100     3    1    140
## 54    Just_Right_Crunchy__Nuggets    4    110     2    1    170
## 55                            Kix    4    110     2    1    260
## 56              Nutri-grain_Wheat    4     90     3    0    170
## 57                     Product_19    4    100     3    0    320
## 58                      Rice_Chex    4    110     1    0    240
## 59                  Rice_Krispies    4    110     2    0    290
## 60                      Special_K    4    110     6    0    230
## 61              Total_Corn_Flakes    4    110     2    1    200
## 62              Total_Whole_Grain    4    100     3    1    200
## 63                        Triples    4    110     2    1    250
## 64                     Wheat_Chex    4    100     3    1    230
## 65                       Wheaties    4    100     3    1    200
## 66            Frosted_Mini-Wheats    5    100     3    0      0
## 67                          Maypo    5    100     4    1      0
## 68                    Puffed_Rice    5     50     1    0      0
## 69                   Puffed_Wheat    5     50     2    0      0
```

```
## 70                          Raisin_Squares        5        90        2   0       0
## 71                          Shredded_Wheat        5        80        2   0       0
## 72                  Shredded_Wheat_'n'Bran        5        90        3   0       0
## 73                Shredded_Wheat_spoon_size        5        90        3   0       0
## 74                  Strawberry_Fruit_Wheats        5        90        2   0      15
##    fiber sugars potass   rating
## 1   10.0      6    280 68.40297
## 2    9.0      5    320 59.42551
## 3   14.0      0    330 93.70491
## 4    2.0      8    135 33.98368
## 5    2.0      8    100 37.03856
## 6    2.0      7    105 40.40021
## 7    4.0      7    160 40.44877
## 8    2.0     10    120 36.17620
## 9    5.0     10    200 40.91705
## 10   5.0     12    190 41.01549
## 11   3.0      4    100 45.81172
## 12   2.0      9     95 36.47151
## 13   2.0      6     95 45.32807
## 14   3.0     11    170 37.13686
## 15   3.0     11    170 34.13976
## 16   3.0     13    160 30.31335
## 17   3.0      7    130 40.69232
## 18   1.5     10    120 30.45084
## 19   6.0     14    260 37.84059
## 20   2.0      6    110 49.51187
## 21   5.0     12    240 39.25920
## 22   2.5      8    140 39.70340
## 23   4.0     14    230 28.59278
## 24   1.5     10     70 29.50954
## 25   1.0     14     30 33.17409
## 26   0.0     12     35 18.04285
## 27   0.0      9     45 19.82357
## 28   0.0     13     55 22.73645
## 29   1.0     12     20 35.78279
## 30   0.0     13     65 22.39651
## 31   1.0     13     30 32.20758
## 32   1.0     11     25 31.43597
## 33   0.0     12     25 28.02576
## 34   0.0     15     40 35.25244
## 35   0.0      9     45 23.80404
## 36   0.0     11     35 28.74241
## 37   1.0     11     45 21.87129
## 38   1.5     10     90 31.07222
## 39   0.0     12     55 26.73451
## 40   2.0      6     90 40.10596
## 41   0.0      9     40 29.92429
## 42   1.0     15     40 31.23005
## 43   0.0     12     25 27.75330
## 44   1.0      8     60 36.18756
## 45   4.0      6    125 49.12025
## 46   5.0      5    190 53.31381
## 47   2.0      1    105 50.76500
## 48   0.0      3     25 41.44502
```

```
## 49   1.0       2      35 45.86332
## 50   1.0       3      30 46.89564
## 51   1.0       5      80 44.33086
## 52   3.0       3      90 53.37101
## 53   3.0       5      85 52.07690
## 54   1.0       6      60 36.52368
## 55   0.0       3      40 39.24111
## 56   3.0       2      90 59.64284
## 57   1.0       3      45 41.50354
## 58   0.0       2      30 41.99893
## 59   0.0       3      35 40.56016
## 60   1.0       3      55 53.13132
## 61   0.0       3      35 38.83975
## 62   3.0       3     110 46.65884
## 63   0.0       3      60 39.10617
## 64   3.0       3     115 49.78744
## 65   3.0       3     110 51.59219
## 66   3.0       7     100 58.34514
## 67   0.0       3      95 54.85092
## 68   0.0       0      15 60.75611
## 69   1.0       0      50 63.00565
## 70   2.0       6     110 55.33314
## 71   3.0       0      95 68.23588
## 72   4.0       0     140 74.47295
## 73   3.0       0     120 72.80179
## 74   3.0       5      90 59.36399
```

```
healthy_cluster <- 5
cereals_clean %>%
  filter(Cluster == healthy_cluster) %>%
  select(name, calories, protein, fat, sodium, fiber, sugars, potass, rating) %>%
  arrange(sugars, sodium, desc(fiber), desc(rating))
```

```
##                        name calories protein fat sodium fiber sugars potass
## 1     Shredded_Wheat_'n'Bran       90       3   0      0     4      0    140
## 2 Shredded_Wheat_spoon_size       90       3   0      0     3      0    120
## 3             Shredded_Wheat       80       2   0      0     3      0     95
## 4                Puffed_Wheat       50       2   0      0     1      0     50
## 5                 Puffed_Rice       50       1   0      0     0      0     15
## 6                       Maypo      100       4   1      0     0      3     95
## 7    Strawberry_Fruit_Wheats       90       2   0     15     3      5     90
## 8              Raisin_Squares       90       2   0      0     2      6    110
## 9         Frosted_Mini-Wheats      100       3   0      0     3      7    100
##     rating
## 1 74.47295
## 2 72.80179
## 3 68.23588
## 4 63.00565
## 5 60.75611
## 6 54.85092
## 7 59.36399
## 8 55.33314
## 9 58.34514
```

Cluster 5 has shredded wheat, puffed wheat, puffed rice, raisin squares, and frosted mini wheats. This cluster has low average calories, very low fat, low sodium, low suger, and a strong average rating. Cluster 1 has a very strong health profile but only contrains three cereals. Cluster 5 is def a more practical choice for the elementary school cafetaria.

##We have to check if these clusters are stable, or did they only happen because of this exact dataset. We need to split the cereals into two parts which is partrition a and partition b. Partition a=used to build clusters and partitiion b=used to test if the clusters still make sense

```
set.seed(123)
X <- cereals_clean %>%
  select(where(is.numeric), -any_of("Cluster"))
n <- nrow(X)
A_index <- sample(1:n, size = floor(0.5 * n))
B_index <- setdiff(1:n, A_index)

A <- X[A_index, ]
B <- X[B_index, ]
A_scaled <- scale(A)
B_scaled <- scale(
  B,
  center = attr(A_scaled, "scaled:center"),
  scale = attr(A_scaled, "scaled:scale")
)
agnes_A <- agnes(A_scaled, method = "ward")
hc_A <- as.hclust(agnes_A)

A_clusters <- cutree(hc_A, k = 5)
centroids <- aggregate(A_scaled, by = list(cluster = A_clusters), mean)
centroids <- as.matrix(centroids[, -1])
B_assigned <- apply(as.matrix(B_scaled), 1, function(row) {
  distances <- apply(centroids, 1, function(center) {
    sum((row - center)^2)
  })
  which.min(distances)
})
comparison <- table(
  B_Assigned_From_A = B_assigned,
  Full_Data_Cluster = cereals_clean$Cluster[B_index]
)

comparison
```

```
##                   Full_Data_Cluster
## B_Assigned_From_A 1 2 3 4 5
##                 1 3 4 1 4 0
##                 2 0 4 0 0 0
##                 3 0 0 9 0 0
##                 4 0 0 0 7 0
##                 5 0 0 0 0 5
```

```
consistency <- sum(apply(comparison, 1, max)) / sum(comparison)

consistency
```

```
## [1] 0.7837838
```

##The consistency score is 78.4%, meaning the clusters are moderately stable. For a small dataset this would be acceptable. Cluster 5's records stayed together in the stability check.

normalization question and final interpretation

Clustering with Euclidean distance requires the data to be normalized because Euclidean distance is scale dependent. Variables such as sodium and potassium have values of a magnitude much larger than fat, protein, and fiber so they would dominate the clustering results if not normalized. The clusters should be created with the normalized data but interpreted with the original values. Cluster 5 is the best practical healthy cereal group for the school cafeteria with low calories, very low fat, almost no sodium, low sugar and a good rating and provides more variety than Cluster 1.